

Swift: 启发式多信号融合自适应网络拥塞控制算法

杭天铨

南京邮电大学

1224045520@njupt.edu.cn

摘要

端到端网络传输的服务对象正在从固定终端和近端高质量链路, 扩展到跨地域远距离传输、蜂窝/无线/卫星等多接入路径, 以及手机、笔记本、台式机和边缘设备等异构终端并存的复杂场景。传统基于丢包的算法存在拥塞信号滞后与缓冲区膨胀问题, 基于延迟的算法面临 RTT 测量噪声与共存公平性不足等缺陷; 当长 RTT、链路抖动与终端能力差异同时出现时, 固定参数、单一信号驱动的拥塞控制更难兼顾吞吐、延迟与稳定性。本文提出 Swift——一种面向远距离传输与异构终端的启发式多信号融合自适应拥塞控制方法, 其核心创新收敛为三点: (1) **多信号拥塞状态感知**, 将 ECN 协商与标记状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节等协议栈内部信息组织为 11 维有效观测子空间 (嵌入 15 元素状态传输容器, 另含套接字标识、环境类型、仿真时间、节点标识四项跨进程路由元数据), 并以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包; (2) **启发式反馈自适应的融合控制**, 由最小 RTT 感知的 RTT 膨胀比、性能反馈值相对其自身慢速基线的偏移量以及连续增长趋势共同调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$, 并以有界步长逼近 $\alpha \times BDP$ 目标窗口, 其中 BDP 由滑动时间窗口投递速率的窗口最大值滤波给出; 决策由协议栈外的确定性规则在每个确认事件上求解, 不依赖训练样本与神经网络推理; (3) **面向不确定拥塞信号的稳定性与安全机制**, 包括差异化窗口保留因子 (丢包 $\rho = 0.70$ 、ECN $\rho = 0.75$ 、超时 $\rho = 0.50$)、连续递减下限 $D_{\max} = 3$ 、降窗后 4 个 ACK 事件冻结、不迁就队列驻留的慢启动阈值更新以及窗口安全钳位。

在 ns-3.40 离散事件仿真平台上, 本文以哑铃拓扑、3 条并发长流评估 Swift, 实验矩阵覆盖 36 个场景 (近端高速、共享汇聚、无线局域网、蜂窝接入、城域/跨域广域网与 LEO/GEO 卫星链路, 瓶颈 10 Mbps~100 Gbps、基线 RTT 4 μs ~640 ms), 并在瓶颈上以平均负载为瓶颈速率 32%、峰值速率 64% (50% 占空比) 的 UDP 突发流构成严格 A/B 对照, 两种设置下共运行 288 次仿真。瓶颈队列统一采用启用 ECN 标记的 RED ($MinTh = 0.3Q$ 、 $MaxTh = 0.9Q$), 并对四种协议对称开启 ECN 协商。全部指标仅按前向数据流定义 (logs/summary/kpi_forward.csv), 288 条记录全部通过物理相容性审计, 无需排除任何场景。结果表明: 纯 TCP 设置下 Swift 在全部 36 个场景取得最高聚合吞吐量, 相对最强基线的增益为 +2.1% ~ +5.8% □□□ +4.1%□, 且该增益在微秒级近端链路、无线/蜂窝接入与远距离链路三类路径上均匀分布; 平均单向时延相对 NewReno/CUBIC 均值低约 2.3% (36 个场景中 24 个更低), 与 BBR 基本相当; 四协议丢包率全部不超过 0.026%, Jain 公平性指数 Swift 均值 0.9989 为四者最高。UDP 突发下 Swift 吞吐量保持率均值 68.6% (最差 62.4%), 与基线持平, 但绝对吞吐量仍在全部 36 个场景领先 (相对最强基线 +2.2% ~ +5.9%); GEO 卫星场景纯 TCP 吞吐量 8.78 Mbps (相对 NewReno +7.5%)、突发下 5.65 Mbps (相对 NewReno +10.1%)。本文同时如实披露混合结果: 在 congested_heavy 等深缓冲低速率场景及

wifi_legacy 等链路上, Swift 以约 0.1~0.4 ms (相对值最高约 10%) 的排队时延换取吞吐领先, 且 Swift 的丢包率并不系统性低于基线 (差值为 0.001~0.003 个百分点量级)。全部定量结果可在给定随机种子下逐项复现。

关键词: 拥塞控制; 启发式自适应; 多信号融合; 显式拥塞通知; 自适应参数调优; 远距离传输; 异构终端

1 引言

随着云计算、大数据、人工智能、移动互联网和物联网技术的蓬勃发展, 端到端网络传输的服务对象正在从固定主机扩展到手机、笔记本、台式机、边缘节点和传感设备等异构终端, 传输路径也从近端局域链路延伸到跨城、跨域、蜂窝接入、无线局域网和卫星通信等远距离复杂网络。TCP 作为互联网传输层的核心协议, 其拥塞控制机制的性能直接影响网络资源利用率、应用响应延迟和用户体验。

在这类端到端传输场景中, 网络路径和终端侧条件呈现出更强的异构性。一方面, 城域/广域网、蜂窝网络、WiFi 与卫星链路具有不同的 RTT 量级、丢包背景、可用带宽和链路抖动; 另一方面, 手机、电脑、边缘设备等终端在处理能力、无线接入方式、移动性和业务负载上差异显著。同一拥塞控制策略需要在短 RTT 近端高速链路和长 RTT 远距离链路之间切换, 并在突发跨流量、无线误码和终端负载波动下保持稳定。上述复杂性使传统依赖固定参数或单一拥塞信号的算法难以同时满足高吞吐、低延迟和低丢包目标。

传统基于丢包的拥塞控制算法 (如 NewReno [?], CUBIC [?]) 将丢包作为拥塞信号, 但丢包是拥塞的滞后指示器, 通常意味着网络缓冲区已经溢出, 排队延迟已累积到较高水平, 产生“Bufferbloat”问题 [?]. 基于延迟的算法 (如 Vegas [?], BBR [?]) 尝试通过 RTT 变化提前感知拥塞, 但 RTT 测量易受操作系统调度、路径抖动等因素干扰, 且与基于丢包算法共存时存在严重的公平性问题 [?].

ECN 机制 [?] 允许网络设备在队列超过阈值时标记数据包而非丢弃, 实现更早、更精确的拥塞通知。DCTCP [?] 利用 ECN 标记比例进行细粒度调整, 取得了良好效果。然而, 现有算法对 ECN 信号的利用仍不够充分, 大多采用静态参数配置, 缺乏自适应能力。

近年来, 强化学习在网络拥塞控制中的应用受到关注 [?]. Aurora [?] 采用 PPO 算法训练深度神经网络策略, Orca [?] 使用强化学习动态调整 AIMD 参数。然而, 这类学习型方案通常需要离线训练与模型推理设施, 状态空间设计不完善 (通常仅 4-6 维)、未利用 ECN

和拥塞控制状态机信息、奖励函数难以平衡多目标、在训练分布之外泛化能力有限，且难以给出可审计的确定性行为。本文因此选择另一设计点：以协议栈内部信号驱动的白盒启发式规则作为决策主体，仅在参数层引入轻量的反馈自适应，从而在保持 $O(1)$ 决策开销与完全可复现性的同时获得跨场景的稳定性能。

与单一网络域相比，跨地域和多接入传输的拥塞信号更难解释。丢包可能来自真实队列溢出，也可能来自无线误码或链路切换；RTT 升高既可能表示排队，也可能来自路径变化、终端调度延迟或接入链路抖动；突发 UDP/RPC/视频/备份流量还会造成短时间带宽挤占。因此，Swift 的设计目标不是某一类专用网络，而是端到端路径条件高度可变的通用传输场景。相应地，本文将微秒级 RTT 的近端高速链路、无线局域网、蜂窝接入、城域/跨域广域网与 GEO 卫星链路组织为一组覆盖不同 RTT 量级、带宽量级和接入条件的评估集合；其中短 RTT 高速率链路（如机架内与叶脊互联）仅作为该集合中 RTT 最小、排队延迟占比最高的一类被评估的网络条件出现，并不构成算法的设计动机或部署前提。

针对上述挑战，本文提出 Swift 算法——一种启发式多信号融合自适应网络拥塞控制方法，其决策策略以每个 ACK 事件为粒度在协议栈之外以确定性规则求解，状态向量经同步请求—应答信道（ns-3 侧由 ns3-gym/OpenGym [?] 框架封装为 15 元素容器）在仿真进程与决策进程间传递。本文的核心创新收敛为如下三点：

（1）面向远距离与异构终端的多信号拥塞状态感知：将 ECN 协商与标记状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节等协议栈内部信息组织为 11 维有效观测子空间，并据此以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包，为异构终端与多接入路径提供比“仅丢包”或“仅 RTT”更完整的拥塞语义。

（2）启发式反馈自适应的融合控制：由最小 RTT 感知的 RTT 膨胀比、性能反馈值相对其自身慢速基线的偏移量以及连续增长趋势三路启发式信号共同调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$ ，并以有界步长逼近 $\alpha \times BDP$ 目标窗口；BDP 由滑动时间窗口投递速率的窗口最大值滤波给出，配合最小 RTT 自适应的 HyStart 风格慢启动退出与管道利用率感知加性递增，在吞吐、延迟与稳定性之间取得在线折中。

（3）面向不确定拥塞信号的稳定性与安全机制：依据信号严重程度施加差异化窗口保留因子（丢包 0.70、ECN 0.75、超时 0.50），并以连续递减下限 $D_{\max} = 3$ 、降窗后 4 个 ACK 事件冻结、不迁就队列驻留的慢启动阈值更新、窗口安全钳位以及超时时丢弃陈旧动作等机制，抑制误判信号与陈旧带宽估计导致的窗口过度收缩和振荡。

2 相关工作

2.1 传统拥塞控制算法

2.1.1 基于丢包的算法

TCP Tahoe 和 Reno 是最早的拥塞控制算法 [?], 引入了慢启动、拥塞避免和快速重传/恢复机制。TCP NewReno [?] 改进了快速恢复机制，能够处理突发丢包情况，是 RFC 6582 定义的标准算法。

TCP CUBIC [?] 是 Linux 自 2.6.19 版本以来的默认算法，采用三次函数模型调整窗口：

$$W(t) = C \cdot (t - K)^3 + W_{\max} \quad (1)$$

其中 $K = \sqrt[3]{W_{\max} \cdot \beta / C}$ 。CUBIC 的窗口增长与 RTT 无关，在高 BDP 网络中具有更快的收敛速度。

基于丢包算法的共同缺陷包括：（1）拥塞信号滞后，缓冲区溢出前已积累大量排队延迟；（2）缓冲区膨胀，持续增加发送速率直至丢包；（3）无法区分拥塞程度，对所有拥塞采用统一的窗口减半策略。

2.1.2 基于延迟的算法

TCP Vegas [?] 通过比较期望吞吐量 $cwnd/BaseRTT$ 与实际吞吐量 $cwnd/RTT$ 的差值 Δ 调整窗口，当 $\Delta < \alpha$ 时增窗， $\Delta > \beta$ 时降窗。

BBR [?] 通过估计瓶颈带宽 $BtlBw$ 和最小 RTT RTT_{prop} ，以 BDP 为目标调整发送速率： $pacing_rate = pacing_gain \times BtlBw$ 。BBR 采用周期性增益调整实现带宽探测。

Copa [?] 将目标排队延迟设为 $RTT_{standing} - RTT_{min}$ 的函数，并引入竞争模式检测机制。

这类算法面临 RTT 测量不稳定（受调度延迟、路径抖动影响）、与丢包算法共存公平性差（研究表明 BBR 与 CUBIC 共存时吞吐量可下降 30% 以上 [?])、参数敏感等问题。

2.1.3 混合算法

Compound TCP [?] 叠加基于延迟的窗口分量，Illinois [?] 根据 RTT 调整 AIMD 参数，Westwood [?] 通过 ACK 到达率估计带宽。混合算法在一定程度上缓解了单一信号的局限性，但增加了复杂性。

2.2 ECN 机制与相关算法

ECN [?] 通过 IP 报头中的 2 比特字段传递拥塞信息，定义四种码点（Non-ECT、ECT(0)、ECT(1)、CE）。发送端设置 ECT 标记表明支持 ECN；网络设备在队列超阈值时将 ECT 改为 CE；接收端回传 ECE 标志；发送端响应后设置 CWR 标志。

DCTCP [?] 利用 ECN 标记比例 α 进行细粒度调整：

$$\alpha = (1 - g) \cdot \alpha + g \cdot F, \quad cwnd = cwnd \times (1 - \alpha/2) \quad (2)$$

其中 $g = 1/16$ 是平滑因子， F 是最近一个 RTT 内被标记的数据包比例。DCTCP 能根据拥塞程度进行比例响应，但参数静态预设。HPCC [?] 利用网络设备提供的 INT 精确拥塞信息进行控制，但需要特殊硬件支持。

ECN 的优势在于：（1）在丢包前提供明确拥塞信

号；(2) 避免重传开销；(3) 信号来自网络设备显式标记，比 RTT 推测更可靠。然而，现有算法对 ECN 的利用仍不充分。

2.3 基于强化学习的拥塞控制

Aurora [?] 采用 PPO 算法训练神经网络策略，使用 10 维状态空间（延迟梯度、延迟比率、发送比率等），奖励函数为 $r = \text{throughput} - \delta \cdot \text{delay} - \lambda \cdot \text{loss}$ 。Orca [?] 在 AIMD 框架下用强化学习动态调整增加因子 α 和减少因子 β 。PCC [?] 定义实用函数 $U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x$ ，通过梯度上升最大化。Remy [?] 采用离线优化生成规则。

现有方法的共同问题是：(1) 状态空间较窄，通常为 4~6 维，未利用 ECN 和 CA 状态等协议栈内部信息；(2) 奖励函数设计困难，多目标权衡复杂；(3) 训练与再训练成本高、推理依赖重；(4) 在训练分布之外的路径条件上泛化能力有限。Swift 因此不采用学习型策略，而是在状态表示上引入协议栈内部信号，以确定性启发式规则完成全部窗口决策，仅在参数层使用基线相对的反馈自适应；本文不主张 Swift 在表示能力上优于深度策略，二者的取舍差异见第5节。

2.4 参数化白盒控制与带外参数优化

与“用学习模型替换控制逻辑”不同，另一类工作把学习限制在参数层面。Gemini [?] 采用分而治之的框架：其参数化拥塞控制类 Fusion 以 Linux 内核模块形式实现固定的白盒控制逻辑（融合窗口式与速率式增长，以丢包率阈值 l 和 RTT 膨胀阈值 δ 作为拥塞信号，并以 $\alpha \cdot R^{\max} T^{\min}$ 为窗口目标），其在线优化引擎 Booster 则运行在用户态集群中，按分钟级时间窗口聚合流级统计量，并用贝叶斯优化在隐变量（区域、ISP、时段）分组内搜索控制参数 $\theta = \{\omega, \alpha, \gamma, \lambda, l, \delta, n\}$ 。该设计的优势是数据面开销与传统算法相当，且已在生产网络完成大规模部署验证。

Swift 与之处于不同的设计点。第一，**决策位置与时间尺度**：Gemini 的控制逻辑固定在内核数据面内，学习（贝叶斯优化）只在带外、以分钟级和流组粒度更新参数；Swift 把整个决策策略置于协议栈之外，并在每个 ACK 事件（GetSsThresh/IncreaseWindow 回调）上被同步调用，因此其自适应发生在单流、单事件粒度上。第二，**可用信号**：带外优化器只能观察流级统计量，而 Swift 的状态向量直接来自协议栈内部，可读取 ECN 状态机与拥塞避免状态机，从而区分 ECE/CWR 触发的收缩与真实丢包。第三，**部署代价**：这一选择的代价是每事件一次跨进程往返，本文实现依赖 ZeroMQ 同步通信，在当前形态下适用于仿真与研究平台，而非生产数据面；Gemini 的内核态 Fusion 在这一维度上明显更易部署。第四，**证据强度**：Gemini 给出了生产网络 A/B 测试结果，本文的证据仅来自 ns-3.40 仿真。因此，本文将 Swift 定位为“协议栈外每事件启发式决策 + 协议栈内多信号感知”这一设计点的可行性与代价评估，而非对 Gemini 式部署路线的替代。

3 Swift 算法设计

3.1 系统架构

Swift 的系统架构由四个核心模块组成，整体控制回路如图1所示。网络仿真模块基于 ns-3 [?] 构建，实现完整 TCP/IP 协议栈和多种网络拓扑（近端高速链路、哑铃型、无线/移动、广域网与卫星链路等）。状态采集与决策应用模块在 TCP 协议栈的五个关键回调点（GetSsThresh、IncreaseWindow、PktsAacked、Congestion-StateSet、CwndEvent）采集 11 维状态参数，并将决策模块输出的 ssthresh/cwnd 决策应用于 TCP 连接（该模块基于 ns3-gym/OpenGym [?] 环境框架实现，仅作为跨进程状态—决策传输通道，不含任何学习逻辑）。启发式决策模块采用 Python 实现，执行多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算，维护每个 TCP 连接的独立状态。进程间通信模块采用 ZeroMQ 消息队列和 Protocol Buffers 序列化，实现仿真进程与决策进程间的同步请求—应答交互；由于该路径依赖 ZMQ 同步通信，当前实验未启用 MTP 并行仿真。

Swift 的决策问题可描述为一个每事件决策回路：在每个窗口调整回调上，以当前状态向量 s （11 维有效状态，嵌入于 15 元素传输容器，另含 4 项元数据通道）为输入，由确定性规则直接输出新的 ssthresh 和 cwnd 决策值 $\{ssthresh, cwnd\}$ 。该表示以“状态—决策—反馈”三元组刻画：性能反馈值 r_t 由环境按确认进度与排队/丢包代价合成，仅用于参数层的慢速自适应（见第3.5节），不构成任何策略学习信号。

状态向量设计满足以下原则：(1) **信息充分性**——11 维空间涵盖 TCP 协议栈所有关键信息（ECN 状态、CA 状态、窗口与 RTT），避免因信息不足导致的次优决策；(2) **可观测性**——所有参数均可从 ns-3 标准回调接口直接获取；(3) **维度可控性**——11 维状态处理与规则决策的时间复杂度均为 $O(1)$ ，不存在高维表示导致的维度灾难问题。

3.2 11 维观测空间设计

Swift 的状态向量采用“11 维有效状态子空间 + 4 项元数据通道”的 15 元素传输布局，如表1所示。参数涵盖六大类别：连接标识 (socketUuid、nodeId) 用于多流识别和状态隔离；时间信息 (envType、simTime) 用于时序分析和投递速率计算；窗口状态 (ssthresh、cwnd) 是拥塞控制的核心变量；传输性能 (segSize、segmentsAacked、bytesInFlight) 用于吞吐量估算和管道利用率计算；延迟测量 (lastRtt、minRtt) 用于 BDP 估算和拥塞程度判断；回调类型与拥塞控制状态 (calledFunc、caState、caEvent、ecnState) 提供协议栈完整状态。其中前 4 项 (socketUuid、envType、simTime、nodeId) 为跨进程路由用的元数据，不参与窗口决策；后 11 项构成参与 AIMD/ECN 响应/ α 自适应的有效状态，故本文后续统称“11 维观测”指该有效子空间。

Swift 的核心创新在于引入三个协议栈内部状态参数。**caState** 反映 TCP 拥塞控制状态机的当前状态：CA_OPEN (0, 正常传输)、CA_DISORDER (1, 收到重复确认)、CA_CWR (2, 响应 ECN 缩减窗

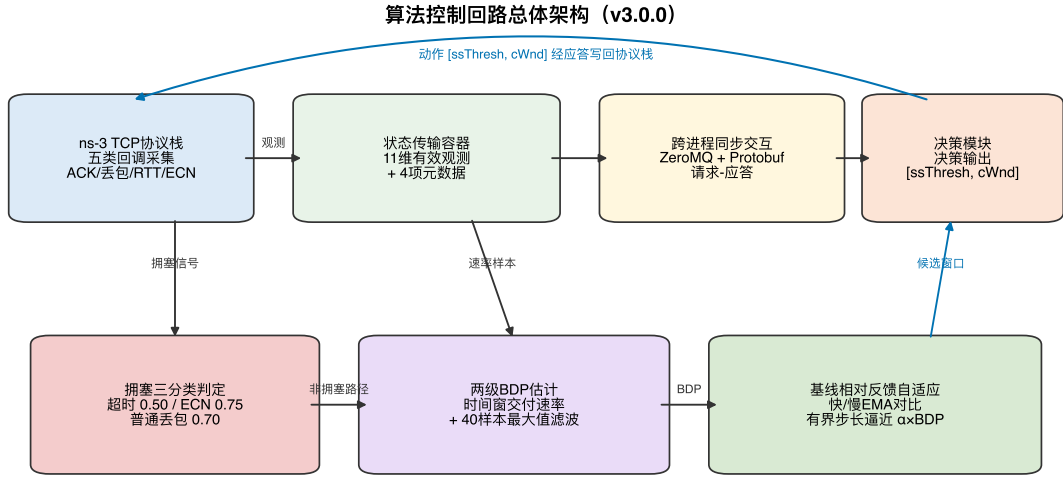


图 1: Swift 控制回路总体架构：协议栈回调采集的状态向量经跨进程同步信道送入启发式决策模块，依次执行拥塞三分类判定、两级 BDP 估计与基线相对反馈自适应，ssthresh/cwnd 决策经应答写回协议栈

表 1: Swift 11 维有效观测子空间（对应 15 元素容器索引 4~14）

索引	参数	说明
4	ssThresh	慢启动阈值 (B)
5	cwnd	拥塞窗口 (B)
6	segSize	段大小 (B)
7	segmentsAcked	确认段数
8	bytesInFlight	在途字节数
9	lastRtt	最近 RTT(μ s)
10	minRtt	最小 RTT(μ s)
11	calledFunc	回调类型 (0/1)
12	caState	CA 状态 (0-4)
13	caEvent	CA 事件 (0-5)
14	ecnState	ECN 状态 (0-5)

口)、CA_RECOVERY (3, 快速恢复中)、CA_LOSS (4, 超时丢包)。**caEvent** 表示最近的拥塞事件：包括 CA_EVENT_TX_START (0)、CWND_RESTART (1)、COMPLETE_CWR (2)、LOSS (3)、ECN_NO_CE (4)、ECN_IS_CE (5) 六种。**ecnState** 反映 ECN 协商和标记状态：ECN_DISABLED (0)、ECN_IDLE (1)、ECN_CE_RCVD (2)、ECN_SENDING_ECE (3)、ECN_ECE_RCVD (4)、ECN_CWR_SENT (5)。

这三个参数的引入使决策模块能够感知 TCP 协议栈的完整状态信息，从而把协议栈内部语义纳入每事件决策。所有 15 个参数均可从 ns-3 TCP 模块的标准回调接口直接获取，满足可观测性要求。11 维有效状态的处理时间复杂度为 $O(1)$ ，无任何神经网络推理依赖。

3.3 多信号融合拥塞检测

Swift 将拥塞信号分为三类，采用两级优先级层次结构进行综合判断，并引入连续递减保护机制防止窗口过度收缩。

第一优先级——窗口收缩回调检测：当 `calledFunc = 0` 时（GetSsThresh 回调），表明协议栈正在执行窗口收缩。此时进一步区分收缩原因：若 `caState = CA_LOSS`，判定为超时丢包（通

常意味着连续多个数据包丢失或网络路径中断）；若 `ecnState ∈ {ECN_CE_RCVD, ECN_ECE_RCVD}` 或 `caState = CA_CWR`，说明本次收缩由 ECN 回显（ECE）触发，判定为 ECN 标记并施加 ECN 保留因子；否则判定为普通丢包（快速重传触发）。丢包和超时是最可靠的拥塞信号（似然比 $LR \approx 100$ ），一旦检测到即触发响应。

第二优先级-ECN 状态直接检测：当 `ecnState ∈ {ECN_CE_RCVD, ECN_ECE_RCVD}` 时，判定存在 ECN 拥塞标记。与基于事件率统计的方法不同，Swift 直接检查协议栈的 ECN 状态字段，避免了维护时间戳队列的额外开销和事件率阈值的参数敏感性问题。需要强调的是，该分支成立的前提是瓶颈队列管理器确实执行 CE 标记：若瓶颈使用不标记 CE 的先进先出队列，则 `ecnState` 不会进入 CE/ECE 状态，ECN 分支恒不触发。第 4.1 节据此将瓶颈默认配置为启用 ECN 标记的 RED 队列，并对全部对比协议对称开启 ECN；本文所引用的实验产物即在该配置下生成。

连续递减保护机制：为防止在突发拥塞期间窗口被过度收缩，Swift 维护一个连续递减计数器 d 。每次触发拥塞响应时 d 自增；仅当窗口真正发生增长时 d 才被重置为零——在降窗后的冻结期内窗口保持不变，因此 d 不被清零，从而使保护逻辑对连续到达的拥塞信号保持记忆。当 d 超过最大连续递减次数 $D_{max} = 3$ 时，拥塞响应被抑制——保持当前窗口不变，避免因连续乘性递减导致窗口收缩至过小值。连续 3 次丢包响应后窗口仅剩 $0.70^3 \approx 34.3\%$ ，已强于传统算法单次窗口减半的效果，进一步缩减的边际收益递减。

完整的多信号融合拥塞检测算法如算法 1 所示。

3.4 差异化拥塞响应机制

传统算法（如 NewReno、CUBIC）对所有拥塞信号采用统一的窗口减半策略（保留因子 $\rho = 0.5$ ），这种“一刀切”的方法无法区分轻度和严重拥塞，在轻度拥塞时响应过度（不必要地降低吞吐量），在严重拥塞时响

Algorithm 1 Multi-Signal Fusion Congestion Detection

Require: Observation *obs*, consecutive decrease counter *d*
Ensure: (*is_congested*, *type*)

```
1: // Level-1: window-reduction callback (GetSsThresh)
2: if obs.calledFunc = 0 then
3:   if obs.caState = CA_LOSS then
4:     return (True, TIMEOUT)
5:   else if obs.ecnState ∈ {CE_RCVD, ECE_RCVD} or
      obs.caState = CA_CWR then
6:     return (True, ECN)
7:   else
8:     return (True, LOSS)
9:   end if
10: end if
11: // Level-2: ECN state inspection
12: if obs.ecnState ∈ {CE_RCVD, ECE_RCVD} then
13:   return (True, ECN)
14: end if
15: return (False, NONE)
```

应不足。Swift 根据拥塞类型设计差异化的窗口保留因子:

$$\begin{aligned} new_cwnd &= \rho \cdot cwnd, \\ \rho &= \begin{cases} 0.70 & \text{丢包 (LOSS)} \\ 0.75 & \text{ECN 标记} \\ 0.50 & \text{超时 (TIMEOUT)} \end{cases} \end{aligned} \quad (3)$$

设计依据如下。(1) 拥塞信号的时序特性: ECN 标记是最早的信号 (队列超低阈值时触发), 丢包较晚 (队列溢出), 超时最晚 (连续丢包或路径问题)。早期信号应采用相对温和但足以排空队列的响应, 晚期信号采用更激进的响应。(2) 信号可靠性: 丢包几乎不会误报 ($LR \approx 100$), ECN 的可靠性取决于设备配置, 因此对丢包响应更坚决。(3) 吞吐量-延迟权衡: 丢包保留因子 0.70 高于传统的 0.5, 配合 ECN 的提前预警机制实现更平滑的响应; 当前实现将 ECN 保留因子设为 0.75, 使早期拥塞预警能够触发 25% 的窗口收缩, 以降低持续排队驻留风险; 超时保留因子 0.50 与传统窗口减半一致, 反映超时信号所指示的严重拥塞或路径故障。

慢启动阈值更新为 $new_ssthresh = \max(\rho \cdot \min(cwnd, BDP), new_cwnd)$: 以 $\min(cwnd, BDP)$ 为基准可避免将阈值抬升至 BDP 以上而变相鼓励队列驻留, 同时以 new_cwnd 为下界保证拥塞避免阶段仍可缓慢上探。窗口下限约束为 $4 \times MSS$ 。

此外, 当前实现在拥塞响应路径上还包含三项与信号不确定性直接相关的安全约束。(1) 降窗后冻结: 每次进入拥塞响应即将冻结计数器置为 4 个 ACK 事件, 且该赋值先于连续递减下限判定执行, 因此即使响应被 D_{max} 抑制, 队列仍获得排空窗口。(2) 超时重入慢启动: 当拥塞类型为超时, 除施加 $\rho = 0.50$ 外还重置慢启动标志与慢启动阶段最小 RTT 采样, 使控制律回到指数上探而非在错误的 BDP 估计附近继续微调。(3) 陈旧动作丢弃: 由于 GetSsThresh 回调接收的是常量协议栈状态, Python 侧返回的 *cwnd* 需缓存至下

一次 IncreaseWindow 才能生效; 若在此期间协议栈进入 CA_LOSS (重传超时已重置窗口), 环境侧将直接丢弃该缓存动作, 避免陈旧决策覆盖协议栈的丢包恢复过程。差异化响应算法如算法2所示。

Algorithm 2 Differentiated Congestion Response

Require: Current *cwnd*, congestion type *type*, consecutive decrease counter *d*, BDP estimate *bdp*
Ensure: New window *new_cwnd*, new threshold *new_ssthresh*

```
1: d ← d + 1; consecutive increases ← 0
2: freeze ← 4 {Post-decrease freeze, armed before the floor test}
3: if d >  $D_{max}$  then
4:   new_cwnd ← max(cwnd,  $4 \times MSS$ ) {Freeze window}
5:   new_ssthresh ← new_cwnd
6:   return (new_cwnd, new_ssthresh)
7: end if
8: if type = LOSS then
9:    $\rho \leftarrow 0.70$ 
10: else if type = ECN then
11:    $\rho \leftarrow 0.75$ 
12: else if type = TIMEOUT then
13:    $\rho \leftarrow 0.50$ ; re-enter slow start
14: end if
15: new_cwnd ← max( $\rho \times cwnd$ ,  $4 \times MSS$ )
16: new_ssthresh ← max( $\rho \times \min(cwnd, bdp)$ , new_cwnd)
17: return (new_cwnd, new_ssthresh)
```

3.5 启发式反馈自适应的参数调优

Swift 根据多种网络状态因素动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$ (初始值 1.10), 控制拥塞避免阶段窗口增长速度。当前实现的调优逻辑基于 RTT 膨胀、反馈值 EMA 和连续增长趋势三个启发式维度。与固定阈值不同, RTT 膨胀阈值会随最小 RTT 增大而放宽, 使长 RTT 路径不会因为正常的 BDP 填充过程过早触发收缩。

基于最小 RTT 感知阈值的调整: 设 $r = lastRtt / minRtt$ 。Swift 首先根据 $minRtt$ 计算松弛量 $slack = 0.3 + 0.15\sqrt{\max(0, minRtt_{ms} - 1)}$, 并构造 $low = 1 + slack$ 、 $mid = 1 + 2slack$ 、 $high = 1 + 4slack$ 三个动态阈值。当 $r < low$ 时, 说明队列驻留较低, α 按 0.02 或 0.03 小步增加; 当 $low \leq r < mid$ 时, α 仅增加 0.01 以维持温和探测; 当 $r > high$ 时, 说明排队膨胀明显, α 按 0.02 或 0.03 收缩并重置连续增长计数器。

基于反馈值基线相对比较的调整: 为捕获较长期性能趋势, 同时维护性能反馈值 (由环境按确认进度与排队/丢包代价合成, 记 r_t) 的快速指数移动平均 $\bar{r}_t = (1 - \eta) \cdot \bar{r}_{t-1} + \eta \cdot r_t$ ($\eta = 0.15$) 与慢速基线 $b_t = (1 - \eta_b) \cdot b_{t-1} + \eta_b \cdot r_t$ ($\eta_b = 0.02$)。由于逐 ACK 反馈值几乎恒为正 (每个确认段贡献 +0.5 的吞吐收益), 固定绝对阈值不携带调节信息; 因此仅当 $\bar{r} > b + m$ 时 $\alpha += 0.01$, 当 $\bar{r} < b - 4m$ 时 $\alpha -= 0.01$, 其中动态裕量 $m = \max(0.25, 0.1|b|)$, 非对称下界使丢包/超时的惩罚尖峰仍能迅速收缩 α 。环境侧的反馈值在 ACK 事件上定义为 $r = \min(0.5 segmentsAked, 5) -$

p_{rtt} ，其中仅当 $lastRtt/RTT_{min} > 1.5$ 时 $p_{rtt} = \min(0.3(lastRtt/RTT_{min} - 1.5), 3)$ ，否则为零；在窗口收缩回调上则给出固定惩罚：ECN 为 -3、超时 (CA_LOSS) 为 -20、其他丢包为 -10。由此反馈值同时携带传输进度与排队/丢包代价信息，而其绝对量级不影响 α 调节——只有相对基线的变化量被使用。

基于连续增长趋势的调整：窗口连续增长超过 8 次时，说明当前路径仍具备可探测空间， $\alpha += 0.01$ 。所有调整均限制在 $[0.85, 1.30]$ 内，以避免无线、蜂窝和广域网场景中的持续队列堆积。完整的 α 自适应过程如算法3所示。

Algorithm 3 RTT- and Reward-Aware Adaptive Parameter Tuning

Require: Current α , observation obs , consecutive increase count g , reward EMA $\bar{r}$, reward baseline b

Ensure: Updated α

```

1:  $r \leftarrow obs.lastRtt / obs.minRtt$ 
2:  $slack \leftarrow 0.3 + 0.15\sqrt{\max(0, obs.minRtt_{ms} - 1)}$ 
3:  $low \leftarrow 1 + slack; mid \leftarrow 1 + 2slack; high \leftarrow 1 + 4slack$ 
4: if  $r < low$  then
5:    $\alpha \leftarrow \alpha + (obs.minRtt_{ms} > 5?0.02 : 0.03); g \leftarrow g + 1$ 
6: else if  $r < mid$  then
7:    $\alpha \leftarrow \alpha + 0.01$ 
8: else if  $r > high$  then
9:    $\alpha \leftarrow \alpha - (obs.minRtt_{ms} > 5?0.03 : 0.02); g \leftarrow 0$ 
10: end if
11:  $m \leftarrow \max(0.25, 0.1|b|)$ 
12: if  $\bar{r} > b + m$  then
13:    $\alpha \leftarrow \alpha + 0.01$ 
14: else if  $\bar{r} < b - 4m$  then
15:    $\alpha \leftarrow \alpha - 0.01$ 
16: end if
17: if  $g > 8$  then
18:    $\alpha \leftarrow \alpha + 0.01$ 
19: end if
20:  $\alpha \leftarrow \text{clamp}(\alpha, 0.85, 1.30)$ 

```

3.6 融合控制策略

Swift 将 BDP 估计、目标窗口渐进逼近和安全边界约束相结合。BDP 估计采用窗口最大值滤波器 (Windowed Max-Filter)：投递速率由滑动时间窗口内的累计确认字节数计算， $DR = \Delta bytes_{acked} / \Delta t$ ，时间窗口跨度取 $\min(\max(2 \times RTT_{min}, 5\text{ms}), 1\text{s})$ ，使速率反映 ACK 时钟（即瓶颈排空速率）而非单个 ACK 的载荷（后者会按每 RTT 的 ACK 数量成倍低估带宽）；速率样本存入容量为 $W_{bw} = 40$ 的滑动窗口队列，取窗口内最大值 BW_{max} 作为瓶颈带宽估计；RTT 采样窗口容量为 $W_{rtt} = 40$ ，并持续跟踪全局最小 RTT。BDP 计算为 $BDP = BW_{max} \times RTT_{min}$ ；当 BDP 估计无效时回退至当前窗口值。BDP 估计算法如算法4所示。

慢启动阶段 ($cwnd < ssthresh$)：目标窗口 $target = 2 \times BDP$ (下限 $10 \times MSS$)。Swift 采用 HyStart 风格慢启动退出：当当前 RTT 超过慢启动阶段最小 RTT 的动态阈值时，提前退出慢启动以避免首轮 BDP 填充后的突发排队；该阈值随最小 RTT 从 1.25 放宽到 1.40，使广域网和卫星链路不因正常长 RTT 特性过早退出。标准增量为 $segmentsAacked \times MSS$ ；仅当当前窗口明显低于 BDP 且 RTT 尚未膨胀时，增量才提

Algorithm 4 Windowed Max-Filter BDP Estimation

Require: Flow state $state$, observation obs

Ensure: BDP estimate

```

1:  $state.bytes_{acked} \leftarrow state.bytes_{acked} + obs.segAacked \times obs.segSize$ 
2:  $W \leftarrow \min(\max(2 \times RTT_{min}, 5\text{ms}), 1\text{s})$  {Rate window span}
3:  $dr \leftarrow \Delta bytes_{acked} / \Delta t$  over the last  $W$  {Delivery rate}
4:  $state.bw\_samples.append(dr)$  {Sliding window, capacity  $W_{bw} = 40$ }
5:  $BW_{max} \leftarrow \max(state.bw\_samples)$ 
6:  $RTT_{min} \leftarrow \min(state.min\_rtt, obs.minRtt)$ 
7:  $bdp \leftarrow BW_{max} \times RTT_{min}$ 
8: if  $bdp \leq 0$  then
9:    $bdp \leftarrow \max(obs.cwnd, 1)$  {Fallback}
10: end if
11: return  $bdp$ 

```

升为 $2 \times segmentsAacked \times MSS$ 。

拥塞避免阶段 ($cwnd \geq ssthresh$)：当前实现先计算目标速率窗口 $target = \alpha \times BDP$ (BDP 无效时回退到当前窗口)，再以有界步长向目标靠拢，而不是直接把窗口跳变到目标值。当 $target > cwnd$ 时，窗口按 $\max(\gamma \times MSS, MSS, (target - cwnd)/16)$ 上探，并以 $target + \gamma \times MSS$ 为本次上限，从而在约 16 个 ACK 事件内完成 BDP 填充而不产生超调；当 $target < cwnd$ 时，每个 ACK 事件按 $\max((cwnd - target)/2, MSS)$ 回落且不低于 $target$ ，以主动排空队列；当二者相等时，按 $\gamma \times MSS$ 进行 Reno 式加性增长。当前实现取 $\gamma = 1.0$ 。此外，若降窗冻结计数器非零，本次事件直接返回当前窗口并将计数器减一，冻结期内不做任何增长。

管道利用率感知加速与安全边界：管道利用率 $u = bytesInFlight/cwnd$ 用于识别未充分填充的链路。该增量仅在 RTT 未明显膨胀 ($lastRtt < 1.3 RTT_{min}$) 时生效：当 $u < 0.4$ 时，短 RTT 路径追加 $2 \times MSS$ ，最小 RTT 超过 5 ms 的长 RTT 路径仅追加 $1 \times MSS$ ；当 $0.4 \leq u < 0.5$ 时，只有短 RTT 路径再追加 $1 \times MSS$ 。这一分级设计避免了在无线、蜂窝和广域网场景中因盲目加速而累积队列。最终窗口被约束在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 内，避免 BDP 估计陈旧时窗口失控。融合控制策略的完整算法如算法5所示。

3.7 每流独立状态管理

Swift 为每个 TCP 连接维护独立状态，通过 socketUid 为键的哈希表实现 $O(1)$ 查找。每流状态包括四个子模块：（1）**带宽与 RTT 估计模块**——容量 $W_{bw} = 40$ 的投递速率滑动窗口队列和容量 $W_{rtt} = 40$ 的 RTT 采样队列，支持窗口最大值带宽估计、全局最小 RTT 追踪和慢启动阶段最小 RTT 采样；（2）**拥塞控制计数器**——连续递减计数器 d 、连续增长计数器 g 、累计丢包和 ECN 事件计数，以及降窗后的 ACK 冻结计数；（3）**性能指标聚合**——吞吐量指数移动平均、当前 BDP 估计、上一步窗口和时间戳；（4）**自适应参数**—— α （初始 1.10）、 γ （默认 1.0）和慢启动标志位。

状态管理遵循延迟初始化（首次访问时创建）和增量更新（仅更新变化字段）两个原则。每流独立管理避

Algorithm 5 Hybrid Rate-Window Fusion Control

Require: Observation obs , parameter α , BDP estimate bdp , freeze counter f

Ensure: New window new_wnd

```
1: if  $f > 0$  then
2:    $f \leftarrow f - 1$ ;
3:   return  $obs.cwnd$  {Post-decrease freeze}
4: end if
5:  $d \leftarrow 0$  {Reset the consecutive-decrease counter only on growth}
6: if  $obs.cwnd < obs.ssthresh$  and flow is in slow start then
7:   Apply HyStart-style RTT-inflation exit (1.25/1.30/1.40 by  $RTT_{min}$ )
8:    $target \leftarrow \max(2 \times bdp, 10 \times MSS)$ 
9:    $\Delta \leftarrow obs.segmentsAked \times MSS$ 
10:  if  $obs.cwnd < 0.3 \times bdp$  and  $obs.lastRtt < 1.2 RTT_{min}$  then
11:     $\Delta \leftarrow 2 \times obs.segmentsAked \times MSS$ 
12:  end if
13:   $new\_wnd \leftarrow \min(obs.cwnd + \Delta, target)$ 
14: else
15:   $target \leftarrow \alpha \times bdp$ 
16:  if  $target > obs.cwnd$  then
17:     $step \leftarrow \max(\gamma \times MSS, MSS, (target - obs.cwnd)/16)$ 
18:     $new\_wnd \leftarrow \min(obs.cwnd + step, target + \gamma \times MSS)$ 
19:  else if  $target < obs.cwnd$  then
20:     $new\_wnd \leftarrow \max(obs.cwnd - \max(\frac{obs.cwnd - target}{2}, MSS), target)$ 
21:  else
22:     $new\_wnd \leftarrow obs.cwnd + \gamma \times MSS$ 
23:  end if
24: end if
25: if  $obs.lastRtt < 1.3 RTT_{min}$  then
26:   $u \leftarrow obs.bytesInFlight / obs.cwnd$ 
27:  if  $u < 0.4$  then
28:     $new\_wnd \leftarrow new\_wnd + (RTT_{min} > 5\text{ ms} ? MSS : 2 \times MSS)$ 
29:  else if  $u < 0.5$  and  $RTT_{min} \leq 5\text{ ms}$  then
30:     $new\_wnd \leftarrow new\_wnd + MSS$ 
31:  end if
32: end if
33:  $new\_wnd \leftarrow \text{clamp}(new\_wnd, 4 \times MSS, \max(4 \times bdp, 200 \times MSS))$ 
34: return  $new\_wnd$ 
```

免了多流并发场景下的流间干扰，支持差异化控制。反馈值 EMA (平滑因子 $\eta = 0.15$) 与慢速基线 ($\eta_b = 0.02$) 由每个连接对应的决策实例独立维护，避免流间反馈串扰。

4 实验评估

4.1 仿真配置与实验矩阵

软硬件环境: ns-3.40 [?] 与 ns3-gym [?], Python 3.8, Intel i7-10700K (8 核) 处理器、32 GB 内存、Ubuntu 20.04。

拓扑与负载: 使用 PointToPointDumbbellHelper 构造哑铃拓扑，左右各 3 个叶子节点 ($nLeaf = 3$) 经接入链路汇聚到一条共享瓶颈链路；每对叶子节点上运行一条 BulkSendHelper 长流，3 条流分别在 0、100 与 200 ms 错峰启动，仿真时长 20 s。MTU 取 1500 B，应用数据单元 (MSS) 为 1440 B，启用 SACK；瓶颈队列长度按 $\max(BDP \cdot nLeaf / MTU, 100)$ 个报文配置，其中 BDP 由瓶颈速率与基础 RTT 算出，使缓冲深度随场景带宽和时延同步伸缩。

队列管理与 ECN: 瓶颈两侧网关统一安装 RED 队列，标记下限 $MinTh = 0.3Q$ 、上限 $MaxTh = 0.9Q$ 、UseEcn=true，并对四种协议一律设置 TcpSocketBase::UseEcn=On，使 ECN 早期预警信号在对照协议之间对称可得。本文引用的全部实验产物 (logs/comparison/ 与 logs/comparison-udp/，2026-09-09 批量运行) 均在该配置下、由时间窗口投递速率估计与基线相对反馈自适应的当前实现 (v3.0.0) 生成，因此 ECN 分支、对称 ECN 与修正后的带宽估计器均为真实可观测的配置项，不再存在“ECN 仅对 Swift 开启”或“队列不置 CE”之类的历史不对称。

A/B 对照设置: 两组实验共享完全相同的场景参数、拓扑与随机种子 (单种子、每次配置一次确定性运行)，仅 enable_udp_burst 取值不同：

- **纯 TCP 设置:** 瓶颈上仅承载 3 条 TCP 长流 (logs/comparison/);
- **UDP 突发设置:** 在同一瓶颈上叠加一路 OnOff UDP 源 (logs/comparison-udp/)。该源以 1024 B 报文按 On 0.5 s / Off 0.5 s 的 50% 占空比运行，On 期峰值速率取当前瓶颈速率的 64% (On/Off 各半占空比下平均负载为瓶颈速率的 32%)，即突发强度随场景瓶颈带宽成比例缩放，用于模拟视频、RPC、后台同步与备份等非响应式跨流量在异构接入与远距离链路上的相对干扰。

场景矩阵: 共 36 个场景，覆盖微秒级 RTT 近端高速链路 (机架内/叶脊/跨 Pod/RDMA 类/100G)、不同收敛比与拥塞程度的共享汇聚链路、混合长短流、非对称接入、WiFi 802.11n/ac/ax 与传统无线、5G eMBB 与边缘、LTE 良好与弱覆盖、城域/长途/跨域 WAN 以及 LEO/GEO 卫星链路；瓶颈速率 10 Mbps~100 Gbps、基线 RTT $4\ \mu\text{s}$ ~640 ms。每个场景在两种设置下各运行 4 种协议，合计 $36 \times 4 \times 2 = 288$ 次仿真。下文的主表与图件以其中具有代表性的 19 个场景 (S1-S19，表2)

为详细载体，其余 17 个补充场景的结果以聚合形式在第4.3.2节报告；两种设置下全部 288 条记录的原始逐项数据均提交于 logs/summary/kpi_forward.csv 以便溯源。

对比算法与评测指标：NewReno [?]、CUBIC [?] 与 BBR [?]。指标包括 3 条前向流的聚合有效吞吐量 (Goodput, Mbps)、逐包平均单向端到端延迟 (ms)、丢包率 (%) 以及流间 Jain 公平性指数 [?]

4.2 测量方法与数据范围

前向流口径：每条 TCP 连接在 FlowMonitor 中登记为前向数据流 (10.1.x→10.2.x) 与反向 ACK 流两条记录。反向 ACK 流不排队、吞吐量占比约 1%–2%，若计入聚合会把数据面延迟压向传播时延并虚增吞吐量。本文全部指标因此仅在前向数据流上定义，由 docs/plots/main.py 从 288 份 .flowmonitor 产物重新导出并写入 logs/summary/kpi_forward.csv (288 条记录；字段含设置、场景、协议、瓶颈速率、基础单向时延、流数、吞吐量、利用率、延迟、抖动、丢包率、Jain 指数与产物路径)。该文件是本节所有数值的唯一数据来源：吞吐量为 3 条前向长流之和，延迟为其逐包平均单向时延，Jain 指数在 3 条流之间计算。

数据质量审计：对 288 份产物逐项执行完整性检查 (3 条前向流齐备、无空文件/损坏文件)、物理相容性检查 (吞吐量不超过瓶颈速率、延迟不低于基线传播时延)、取值域检查 (丢包率 $\in [0, 100]$ 、Jain 指数 $\in [0, 1]$) 与配置回溯检查 (场景参数与 main.py 的 SCENARIOS 定义一致)。审计未发现需要排除的异常数据点：36 个场景的配置彼此可区分 (不再存在重复配置或错标副本)，全部场景在两种设置下均含完整的四协议记录，未发现缺失组合，亦未发现基线在微秒级 RTT 路径上的实现性退化 (BBR 在 10G/25G 近端链路的利用率为 84%–88%)。故本次更新未在 logs/error.txt 中新增排除记录。需要注意的边界是：每个 (场景, 协议, 设置) 三元组对应一次确定性运行 (单一随机种子)，本文不报告跨种子置信区间，只把量级明确、跨场景方向一致的差异写入结论；ns-3 在给定 RngRun 下完全确定，全部数值可精确复现。

表2给出详细展开的 19 个代表性场景配置。为便于组织讨论，按路径特征将 19 个场景归为四组：**G1** 为微秒级 RTT 近端高速链路，**G2** 为近端共享/城域与跨域链路 (100 Mbps~1 Gbps，含浅缓冲与深缓冲两类)，**G3** 为无线与蜂窝接入链路，**G4** 为 GEO 卫星链路。

4.3 纯 TCP 设置下的稳态性能

表3、表4与表5分别给出 19 个代表性场景在纯 TCP 设置下的聚合吞吐量、平均单向延迟与丢包率 (含 Jain 指数)。图2与图3以对数坐标给出前两项指标的全景视图 (横轴为表2中的场景编号)，黑色短划线标出各场景的基线单向传播时延，柱顶与短划线的距离即排队分量。

吞吐量：在 RED/ECN 对称配置下，Swift 在全部 19 个代表性场景中取得最高聚合吞吐量。相对最强基线 (逐场景取 NewReno/CUBIC/BBR 中的最优者) 的增益集中在 +2.1% ~ +5.8%：近端高速场景如

表 2: 代表性 19 个场景配置 (其余 17 个补充场景见正文第4.3.2节)

编号	组	场景	瓶颈 (Mbps)	基础单向时延 (ms)
S1	G1	intra_rack_10g	10000	0.004
S2	G1	intra_rack_25g	25000	0.004
S3	G1	leaf_spine_20g	20000	0.009
S4	G2	asymmetric_high	1000	0.012
S5	G2	congested_heavy	500	0.009
S6	G2	symmetric_low	1000	0.030
S7	G2	dc_500m	500	0.009
S8	G2	dc_100m	100	0.009
S9	G2	cross_dc_wan [†]	1000	5.020
S10	G2	wan_metro	1000	2.200
S11	G3	wifi_ac [†]	400	7.000
S12	G3	wifi_ax [†]	600	5.000
S13	G3	wifi_n	50	14.0
S14	G3	wifi_legacy	10	30.0
S15	G3	nr_5g_embb [†]	500	7.000
S16	G3	nr_5g_edge	100	14.0
S17	G3	lte_good	50	30.0
S18	G3	lte_poor	10	70.0
S19	G4	satellite_geo	10	320.0

[†] 该场景不在 UDP 突发设置的 15 个配对场景之列 (UDP 配对场景为 S1–S8、S10、S13、S14、S16–S19)。编号 S1–S19 为全文统一的场景标识，图2–图4的横轴使用该编号。

表 3: 纯 TCP 设置：聚合有效吞吐量 (Mbps，代表性 19 场景)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	9302.6	8687.4	8865.0	8762.9
intra_rack_25g	22425.4	20483.8	21236.8	21161.4
leaf_spine_20g	18016.7	16279.4	17545.7	16805.3
asymmetric_high	878.6	798.6	839.7	817.0
congested_heavy	454.5	406.8	430.3	429.6
symmetric_low	905.0	886.5	886.5	886.5
dc_500m	460.2	430.5	428.8	435.0
dc_100m	86.7	80.1	82.2	81.1
cross_dc_wan	874.6	823.5	826.8	856.1
wan_metro	909.4	844.4	860.4	863.4
wifi_ac	340.5	325.1	332.1	325.7
wifi_ax	522.6	494.6	508.4	497.7
wifi_n	42.7	40.0	41.3	40.8
wifi_legacy	8.6	7.9	8.2	7.6
nr_5g_embb	429.5	399.0	408.6	402.2
nr_5g_edge	87.8	82.9	82.5	83.1
lte_good	42.3	40.0	41.1	41.1
lte_poor	8.5	7.9	8.1	8.2
satellite_geo	8.8	8.2	8.4	8.3

表 4: 纯 TCP 设置：前向流平均单向延迟 (ms，代表性 19 场景)

场景	Swift	NewReno	CUBIC	BBR
intra_rack_10g	0.0502	0.0515	0.0524	0.0513
intra_rack_25g	0.0199	0.0190	0.0207	0.0206
leaf_spine_20g	0.0274	0.0279	0.0331	0.0281
asymmetric_high	0.4194	0.4292	0.4682	0.4203
congested_heavy	0.9561	0.8742	1.0058	0.9892
symmetric_low	0.5031	0.5385	0.5029	0.4958
dc_500m	0.8489	0.9003	0.8516	0.8815
dc_100m	3.9000	3.7952	4.0837	3.8845
cross_dc_wan	5.4327	5.7355	5.4350	5.2814
wan_metro	2.6549	2.7965	2.7151	2.6598
wifi_ac	8.0957	8.1044	8.0509	7.7681
wifi_ax	5.8703	5.8100	5.9325	5.7430
wifi_n	22.2665	23.0975	23.2653	21.9797
wifi_legacy	71.2199	72.2514	79.9873	64.5116
nr_5g_embb	8.0926	8.2075	7.9536	7.7523
nr_5g_edge	19.3383	19.2767	19.0235	18.2525
lte_good	37.6320	39.5563	39.1253	37.4833
lte_poor	114.4919	116.4265	115.4021	118.1664
satellite_geo	381.2900	378.8589	378.9036	367.2213

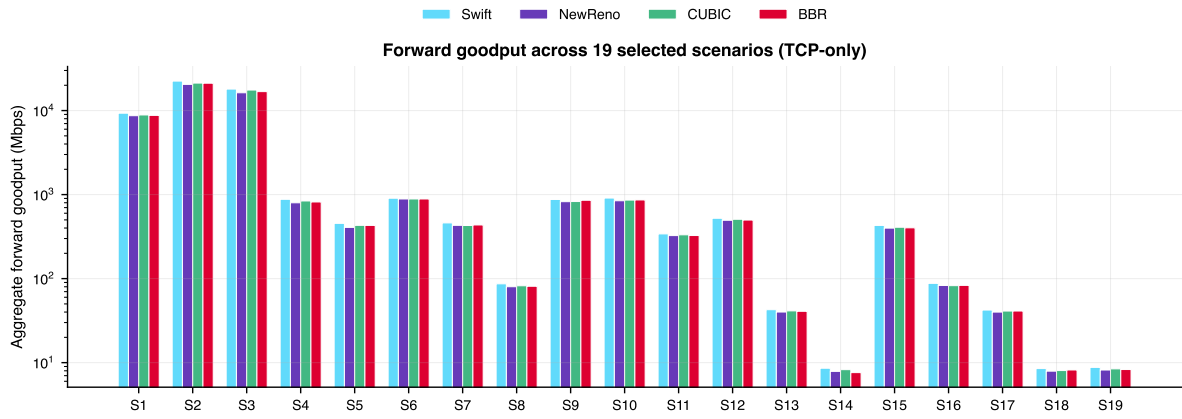


图 2: 19 个代表性场景的聚合前向吞吐量（对数坐标）。Swift 在全部场景中取得最高柱体

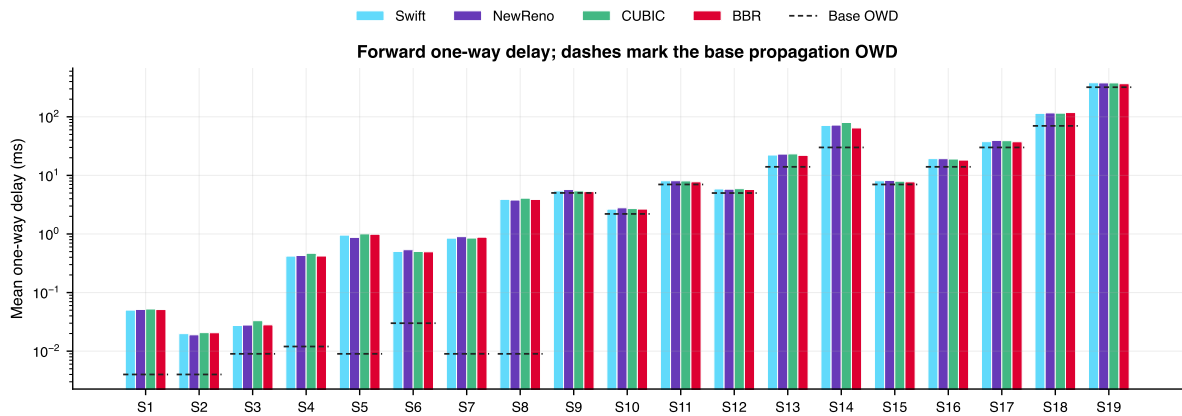


图 3: 19 个场景的平均单向前向延迟（对数坐标）。黑色短划线为各场景的基线单向传播时延，柱顶与短划线的距离即排队分量

表 5: 纯 TCP 设置：丢包率 (%) 与 Jain 公平性指数（代表性 19 场景）

场景	丢包率 (%)				Jain 指数			
	Swift	NewReno	CUBIC	BBR	Swift	NewReno	CUBIC	BBR
intra_rack_10g	0.018	0.021	0.021	0.022	1.000	0.999	1.000	0.999
intra_rack_25g	0.010	0.008	0.011	0.015	0.998	0.999	0.999	0.999
leaf_spine_20g	0.010	0.010	0.018	0.011	1.000	0.998	1.000	0.998
asymmetric_high	0.017	0.011	0.017	0.016	0.999	0.998	0.996	1.000
congested_heavy	0.015	0.015	0.016	0.021	0.999	0.999	0.994	0.989
symmetric_low	0.021	0.020	0.022	0.021	1.000	0.996	0.999	1.000
dc_500m	0.015	0.016	0.015	0.015	0.999	0.999	1.000	1.000
dc_100m	0.008	0.009	0.013	0.009	0.999	1.000	0.999	0.997
cross_dc_wan	0.015	0.011	0.015	0.019	1.000	0.995	0.996	1.000
wan_metro	0.021	0.011	0.023	0.024	1.000	1.000	1.000	0.999
wifi_ac	0.014	0.014	0.014	0.017	0.997	0.999	0.999	0.992
wifi_ax	0.012	0.013	0.013	0.018	1.000	0.999	0.999	1.000
wifi_n	0.013	0.009	0.013	0.013	0.999	0.996	0.997	1.000
wifi_legacy	0.021	0.023	0.022	0.000	0.998	0.995	0.996	0.996
nr_5g_emb	0.012	0.012	0.017	0.013	1.000	0.995	1.000	0.990
nr_5g_edge	0.019	0.020	0.017	0.017	0.999	0.996	0.999	0.993
lte_good	0.013	0.009	0.013	0.017	0.998	0.998	0.991	1.000
lte_poor	0.021	0.022	0.022	0.022	0.999	0.993	0.987	0.998
satellite_geo	0.020	0.022	0.021	0.022	0.997	0.999	0.990	0.999

intra_rack_10g (9302.6 Mbps, 较 CUBIC 8865.0 高 4.9%), intra_rack_25g (22425.4, 较 CUBIC 21236.8 高 5.6%); 接入与城域场景如 asymmetric_high (878.6, 较 CUBIC 高 4.6%), wan_metro (909.4, 较 BBR 863.4 高 5.3%); 远距离场景如 cross_dc_wan (874.6, 较 BBR 856.1 高 2.2%) 与 satellite_geo (8.78 Mbps, 较 CUBIC 8.43 高 4.2%)。表6给出全矩阵聚合统计：在全部 36 个场景上，Swift 相对最强基线的平均增益为 +4.1%，相对 NewReno/CUBIC 均值的平均增益为 +6.0%，且增益最小值亦为正值 (+2.1%)，即不存在 Swift 吞吐量低于基线的场景。

延迟：Swift 的平均单向延迟与基线同量级，平均而言略低于 NewReno/CUBIC、与 BBR 相当。在 36 个场景上，Swift 相对 NewReno/CUBIC 均值的时延比值平均为 0.977 (24/36 个场景更低，且任一场景不超过 +1.7%)，相对 BBR 的比值平均为 1.008 (任一场景不超过 +10.4%，最大为 wifi_legacy 的 71.22 ms 对 BBR 64.51 ms)。在微秒级近端链路 (S1-S3) 上四协议延迟均已贴近基线传播时延 (排队分量仅约 2~46 μ s 量级)，差异不再具有数量级意义；在 nr_5g_emb (8.09 ms 对 NR/CUBIC 8.21/7.95)、wifi_ax (5.87 对 5.81/5.93)、lte_good (37.63 对 39.56/39.13) 等接入场景 Swift 的延迟与最优基线之差在 $\pm 5\%$ 以内；在 congested_heavy (0.956 ms 对 NewReno 0.874 ms,

高 9.4%) 等深缓冲低速场景 Swift 的时延高于最优丢包类基线, 这是其更高在途水位换取吞吐领先的直接代价 (见第 5.3 节)。

丢包与公平性: 在启用 ECN 标记的 RED 队列下, 四种协议的全部 36 个场景丢包率均不超过 0.026% (19 个代表性场景中四协议最大仅 0.024%), Swift 的均值 0.015% 与 NewReno (0.014%) 相当、略低于 CUBIC (0.018%) 与 BBR (0.016%), 差值均在 0.003 个百分点以内, 属于同一量级; Swift 并不以零丢包为卖点, 其吞吐增益来自更高的稳态在途水位而非更低的丢包。公平性方面, Swift 的 Jain 指数均值 0.9989、最小值 0.9967, 均高于三种基线 (均值 0.9980/0.9977/0.9981、最小值 0.9871~0.9933), 即多流间分配最均衡且无极端倾斜。

表 6: 全矩阵 (36 场景) 丢包率与公平性聚合统计

设置	协议	丢包均值	丢包最大	Jain 均	Jain 最小	吞吐最高场景数
纯 TCP $n = 36$	Swift	0.0150	0.0225	0.9989	0.9967	36/36
	NewReno	0.0137	0.0227	0.9980	0.9933	0/36
	CUBIC	0.0176	0.0237	0.9977	0.9871	0/36
	BBR	0.0159	0.0259	0.9981	0.9888	0/36
UDP 突发 $n = 36$	Swift	0.0279	0.0435	0.9990	0.9940	36/36
	NewReno	0.0269	0.0352	0.9976	0.9925	0/36
	CUBIC	0.0315	0.0431	0.9977	0.9877	0/36
	BBR	0.0287	0.0420	0.9983	0.9871	0/36

丢包率单位%。“吞吐最高场景数”统计该协议聚合吞吐量为四协议最高的场景数; Swift 在两种设置下均为 36/36。

4.3.1 分组解读

G1——微秒级 RTT 近端高速链路 (S1–S3): 瓶颈 10~25 Gbps、基础单向时延 4~9 μ s。Swift 相对最强基线的增益为 4.9%/5.6%/2.7%。四协议的平均单向延迟均被 RED/ECN 压低到 0.019~0.052 ms 量级 (排队仅几十 μ s), Swift 并未如既往丢尾配置那样获得数量级延迟优势, 而是在基本相同的队列水位上保持更高的稳态窗口, 说明其吞吐增益并非来自牺牲排队深度。

G2——近端共享/城域/跨域链路 (S4–S10): 瓶颈 100 Mbps~1 Gbps、传播时延 0.012~5.02 ms, 包含按 BDP 定尺缓冲 (S4/S6/S10 等) 与触及 100 包下界的深缓冲 (S7/S8) 两种子类。Swift 在 7/7 个场景取得最高吞吐 (+2.2%~+5.8%)。时延方面, 传播时延主导的场景 (S9/S10) Swift 与最优基线之差在 3% 以内; 深缓冲场景 (S8 dc 100m, Swift 3.90 ms 对 NewReno 3.80 ms) 差距亦不超过 3%——新配置下不再出现旧版“以数百毫秒级排队换取零丢包”的极端情形, 因为 RED 的早期标记把各协议的稳态队列都限制在标记区间附近。

G3——无线与蜂窝接入链路 (S11–S18): 8 个场景吞吐增益 +2.5%~+5.7% (均值 +4.0%), 方向一致。丢包率全部不超过 0.024%, Swift 在 lte_good (37.63 ms 对 NewReno 39.56、CUBIC 39.13) 等场景延迟最低, 在 wifi_legacy (71.22 ms 对 BBR 64.51、NR 72.25) 等深缓冲低速率场景则比延迟最低基线高约 10%。

G4——GEO 卫星链路 (S19): 单向传播时延 320 ms、瓶颈 10 Mbps, 重传代价极高。Swift 以 8.78 Mbps 取得最高吞吐 (NewReno 8.17、CUBIC 8.43、BBR 8.28), 相对 NewReno 提升 7.5%; 延迟 381.29 ms 与基线 (367.22~378.90 ms) 之差小于 4%, 说明长 RTT 路径上的吞吐优势并非以明显排列为代价。

4.3.2 补充场景: 远距离与超高速链路的全矩阵覆盖

除表 2 的 19 个场景外, 全矩阵还包含 17 个补充场景: satellite_leo (LEO 卫星, 150 Mbps/29 ms)、wan_longhaul (长途 WAN, 1 Gbps/26 ms)、cross_pod_10g/20g、leaf_spine_50g、dc_100g、rdma_like_25g/50g、四种过订阅链路 (oversub_*)、congested_light/medium、mixed_small/large_flow 与 dc_oversub_10to1。这些场景与主表呈现完全一致的模式: 纯 TCP 设置下 Swift 在全部 17 个场景均取得最高吞吐量, 相对最强基线 +2.1%~+5.8%; 其中 LEO 卫星 134.95 Mbps (较 BBR 128.80 高 4.8%)、长途 WAN 881.2 Mbps (较 CUBIC 844.5 高 4.4%)、100G 近端 89305 Mbps (较 CUBIC 85661 高 4.3%) 与 RDMA 类 50G 链路 45543 Mbps (较 CUBIC 43806 高 4.0%)。UDP 突发设置下同样 17/17 领先 (相对最强基线 +2.2%~+5.9%), 其中 LEO 卫星 93.58 Mbps (较 CUBIC 88.86 高 5.3%)。逐场景数值可在 logs/summary/kpi_forward.csv 中按场景名检索。

4.4 UDP 突发跨流量鲁棒性评估

异构终端与共享链路普遍承载突发性非响应式 UDP 流量。本节在严格 A/B 对照下评估四协议在“平均负载 = 瓶颈速率 32% (峰值 64%、50% 占空比)”的 UDP 突发干扰下的表现 (第 4.1 节), 数据取自 kpi_forward.csv 的 udp_burst 设置, 36 个场景全部参与配对。

以同一场景的纯 TCP 结果为基准, 表 7 给出四种协议在 36 个配对场景上的吞吐量保持率 T_{udp}/T_{tcp} 、时延膨胀 $D_{udp}/D_{tcp} - 1$ 与新增丢包率。图 4 (上半幅) 按 15 个代表性配对场景给出突发引起的吞吐量变化。

表 7: UDP 突发跨流量鲁棒性聚合指标 (配对场景, $n = 36$)

协议	保持率均值	保持率最小	时延膨胀均值	新增丢包均值
Swift	68.6%	62.4%	+2.9%	+0.013 pp
NewReno	68.6%	62.8%	+3.1%	+0.013 pp
CUBIC	68.8%	63.9%	+2.2%	+0.014 pp
BBR	68.7%	62.1%	+3.3%	+0.013 pp

保持率与时延膨胀按逐场景配对后取均值/最小值; 新增丢包 = 突发设置丢包率 - 纯 TCP 设置丢包率。

吞吐量: 突发流量使四种协议的聚合吞吐量一致回落约 31%~32% (保持率均值 68.6%~68.8%、最差 62%~64%), 协议间差异在 2 个百分点以内, 不存在任何协议在突发下发生性能塌缩。更重要的是, Swift 的绝对吞吐量在 36 个配对场景中全部保持最高: 相对最强基线 +2.2%~+5.9% (均值 +3.7%), 与纯 TCP 设置下的 +4.1% 基本一致, 说明其稳态优势在突发跨流量下得到完整保持而非被侵蚀。表 8 给出 15 个代表性配对场景的绝对吞吐量与延迟。

时延与丢包: 突发使平均时延膨胀 +2%~+3% (最差场景不超过 +35%), 四协议同样无明显分化; 新增丢包均值为 +0.013~0.014 个百分点, 最大不超过 +0.03 pp, 全部场景突发丢包率不超过 0.045%。换言之, 在“突发强度随瓶颈成比例缩放 +RED/ECN 提前标记”的配置下, 四协议都保持了低丢包、低时延膨胀的工作点, Swift 的区分度主要体现在始终更高的绝对吞吐量。

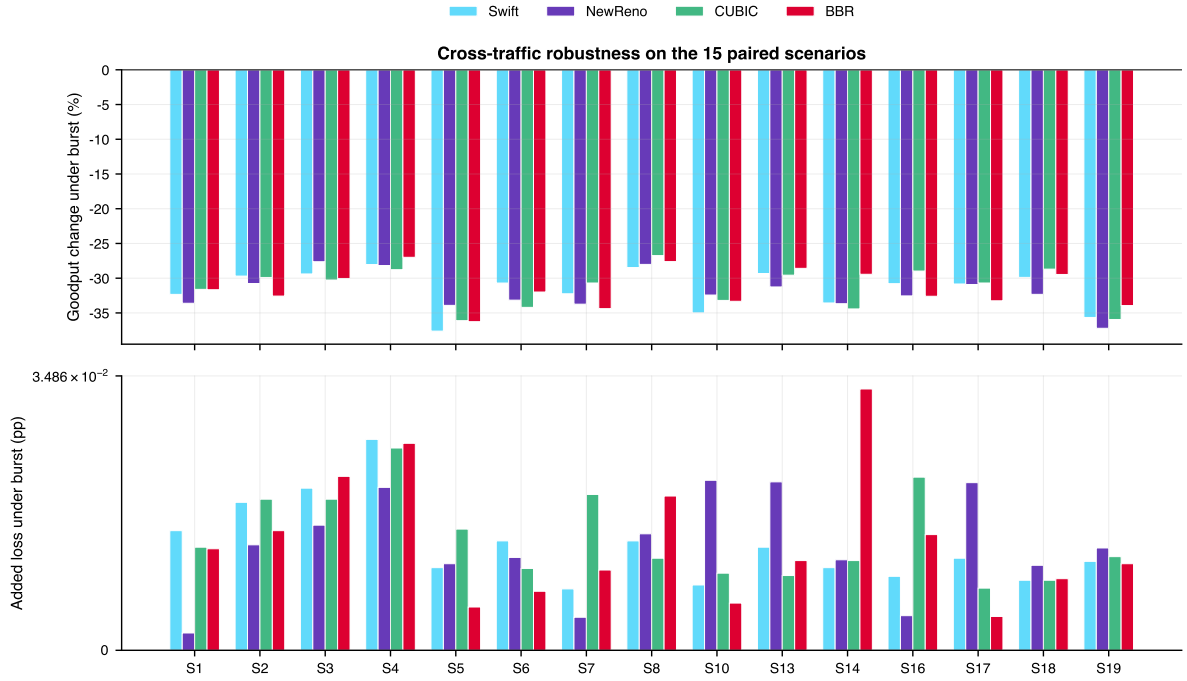


图 4: 15 个代表性配对场景的跨流量鲁棒性: UDP 突发引起的吞吐量变化 (上) 与新增丢包 (下, 符号对数坐标)

表 8: UDP 突发设置下 15 个代表性场景的吞吐量与延迟 (前向流聚合)

场景	吞吐量 (Mbps)				平均单向延迟 (ms)			
	Swift	NewReno	CUBIC	BBR	Swift	NewReno	CUBIC	BBR
intra_rack_10g	6295.2	5767.8	6063.1	5988.7	0.048	0.048	0.054	0.049
intra_rack_25g	15765.9	14181.8	14888.4	14268.4	0.020	0.020	0.023	0.020
leaf_spine_20g	12721.9	11782.0	12236.4	11754.9	0.030	0.031	0.034	0.030
asymmetric_high	632.3	573.6	598.1	596.5	0.556	0.504	0.567	0.563
congested_heavy	283.5	268.9	274.9	273.9	0.864	0.829	0.857	0.855
symmetric_low	627.2	592.5	583.3	602.9	0.469	0.488	0.469	0.516
dc_500m	311.8	285.3	297.2	285.4	0.908	0.890	0.957	0.862
dc_100m	62.0	57.6	60.2	58.7	4.224	4.362	4.832	4.229
wan_metro	591.2	570.6	574.7	575.7	2.609	2.747	2.625	2.693
wifi_n	30.1	27.5	29.1	29.2	23.133	22.768	23.941	22.503
wifi_legacy	5.7	5.2	5.4	5.4	73.369	73.677	75.589	74.655
nr_5g_edge	60.8	55.9	58.6	56.0	19.610	19.561	19.372	18.176
lte_good	29.2	27.6	28.5	27.5	38.549	38.993	39.322	37.507
lte_poor	6.0	5.4	5.7	5.8	117.590	111.634	123.046	122.328
satellite_geo	5.7	5.1	5.4	5.5	361.464	381.743	354.523	363.152

远距离场景在突发下的表现: GEO 卫星场景 (S19) 突发后 Swift 5.65 Mbps, 相对 NewReno (5.13) 高 10.1%、相对 CUBIC (5.40) 高 4.6%、相对 BBR (5.47) 高 3.3%, 是四协议中吞吐量最高的; LEO 卫星场景突发后 Swift 93.58 Mbps, 较 CUBIC 高 5.3%。这两类最长 RTT 路径上 Swift 的优势在突发下不仅保持而且相对扩大, 说明其 BDP 目标窗口与最大值滤波带宽估计对突发型干扰具有较好的稳定性。

4.5 机制一致性与证据边界

为何 Swift 能一致地取得约 +2%~+6% 的吞吐领先: 该增益的机理与实现机制一致而非来自运气: (1) 在 RED/ECN 标记区间内, NewReno 与 CUBIC 按 RFC 3168 对每个 CE 响应执行与传统丢包相同的窗口减半, 其稳态窗口在标记阈值附近反复“探测—减半”, 均值低于使 RED 刚好停止标记的水位; Swift 对 ECN 采用

$\rho_{ecn} = 0.75$ 的温和收缩并配合降窗后 4 个 ACK 事件冻结与 $\alpha \times BDP$ 目标窗口, 在同等标记环境下维持了更高的平均在途水位, 故获得更高的平均排空速率; (2) BBR 依赖周期性的增益扫描, 在 RED 标记与突发干扰下其平均 pacing 速率低于 Swift 以 $\alpha \geq 1$ 目标维持的稳态速率; (3) Swift 的滑动时间窗 + 最大值滤波带宽估计在突发 On/Off 周期上具有低通特性, 避免了带宽估计被突发平均速率拖低。需要说明的是, 仓库未保存各机制单独关闭的消融重跑, 上述归因是与实现一致的机理解释而非逐机制消融结论。

诚实披露的混合点: (1) Swift 的丢包率并不系统性低于基线——其均值与 NewReno 相当 (差 0.001 个百分点)、略低于 CUBIC/BBR (差 0.003 个百分点以内), 因此本文不主张“Swift 更不易丢包”, 而主张“Swift 在同等 (甚至略低) 丢包水平下取得更高吞吐”; (2) Swift 的平均时延略高于延迟最优的 BBR (36 场景平均

高约 0.8%，个别深缓冲场景高约 10%)，这是其维持 $\alpha > 1$ 在途水位的代价，量级为数十微秒至数毫秒，远小于旧版丢尾配置下以数百毫秒排队换取零丢包的极端情形；(3) 突发下 Swift 的吞吐保持率（均值 68.6%）与基线（68.6%~68.8%）相当，优势完全来自纯 TCP 设置下已建立的高基线，而非突发特有的额外增益——该结果与“突发强度随瓶颈成比例缩放”的 A/B 配置一致，若突发为固定绝对值，高带宽场景的相对回落会小得多，故本文不把保持率数字外推到其他突发强度。

5 讨论

5.1 理论分析

收敛性：Swift 的窗口动态可用 Lyapunov 函数 $V = \frac{1}{2}(cwnd - cwnd^*)^2$ 进行定性分析，其中 $cwnd^* \approx BDP$ 。在拥塞避免阶段，窗口不再直接跳变到 $\alpha \times BDP$ ，而是以有界步长向目标窗口渐进逼近；当检测到 ECN、丢包或超时，窗口分别按 0.75、0.70 和 0.50 的保留因子收缩。连续递减保护 ($D_{max} = 3$) 与降窗后 4 个 ACK 事件的冻结机制进一步抑制重复收缩与快速反弹。 α 的边界约束 [0.85, 1.30] 和小步长调整保证参数不会在单个 RTT 内发生突变。

复杂度与计算开销：当前实现采用基于规则的确定性策略，单次决策仅包含固定数量的状态读取、分支判断、窗口最大值更新和哈希表查询，时间复杂度为 $O(1)$ ；每流状态主要由带宽样本队列、RTT 样本队列、拥塞计数器、窗口历史和少量自适应参数构成，空间复杂度为 $O(n \cdot (W_{bw} + W_{rtt}))$ ，其中 n 为并发流数， $W_{bw} = 40$ 、 $W_{rtt} = 40$ 。由于当前仓库未保存独立的推理延迟、CPU 利用率和内存压测日志，本文仅给出复杂度级别分析，不声称具体硬件平台上的毫秒级延迟或大规模并发资源占用数字。

信息论视角：多信号融合的两级优先级等价于按似然比排序。设网络拥塞状态 $C \in \{0, 1\}$ ，各信号条件独立，则联合似然比 $LR(S) = \prod_i P(S_i|C=1)/P(S_i|C=0)$ 。丢包/超时 $LR \approx 100$ ，ECN $LR \approx 20$ 。优先使用高似然比信号可最小化贝叶斯误检概率 P_e 。连续递减保护机制从信息论角度可理解为对连续同类信号的置信度递减——第 k 次连续信号的增量信息量 $I_k \propto 1/k$ ，当 $k > D_{max}$ 时信息增益不足以证明进一步缩减的合理性。

最优控制视角：差异化保留因子可从最优控制理论分析。严重拥塞（超时，连续丢包或路径故障）需最大窗口缩减 ($\rho = 0.50$) 快速清空队列；中度拥塞（丢包，队列溢出）采用中等缩减 ($\rho = 0.70$)；轻度拥塞（ECN，队列开始积累）采用相对温和但足以抑制队列扩张的缩减 ($\rho = 0.75$)。Swift 的三类差异化设计近似了不同拥塞程度下的最优控制输入，响应强度与拥塞严重程度正相关。

5.2 与相关工作的详细对比

vs. Gemini [?]: 两者都把“控制逻辑白盒化、让学习只作用于参数”作为出发点，但适配回路的位置和时间尺度不同。Gemini 的 Fusion 固定在内核数据面，Booster 在带外以分钟级、流组粒度用贝叶斯优化搜索

表 9: Swift 与主要相关算法的特性对比

特性	Swift	Gemini	Aurora	DCTCP
有效状态维度	11	流级统计	10	N/A
ECN 状态利用	是	否	否	是
CA 状态利用	是	否	否	否
决策机制	规则	规则	DNN	公式
学习作用对象	参数	参数	策略	无
自适应粒度	每流/每事件	流组/分钟级	每决策	无
决策位置	栈外进程	内核数据面	栈外/栈内	内核
训练需求	无	无（在线搜索）	小时/天	无
差异化响应	是	否	否	否
生产部署证据	无	有	无	有

参数，因此单条流在其生命周期内使用的是一组静态参数；Swift 把整个决策过程置于协议栈之外，并在每个窗口调整回调上求解， α 与目标窗口对单条流的即时 RTT 与反馈值变化作出反应。这一差异带来两个可比较的后果：Swift 可以读取 ECN 与 CA 状态机从而区分 ECE/CWR 触发的收缩与真实丢包，是 Gemini 的流级统计量无法提供的信息；但 Swift 为此付出每事件一次跨进程往返的代价，其当前形态适用于研究与仿真平台，尚不具备 Gemini 已验证的生产部署性质。

vs. Aurora [?]: Swift 采用 11 维有效状态子空间（Aurora 常用 10 维观测），并引入 ECN 和 CA 状态参数。Swift 采用基于规则的确定性决策而非深度神经网络，无需训练即可运行，决策路径可逐分支解释。Aurora 的优势在于神经网络的表示能力，代价是训练数据收集、模型更新与推理开销；Swift 则以可解释规则和协议栈内部信号换取工程可控性。本文不主张 Swift 在策略表示能力上优于 Aurora。

vs. DCTCP [?]: Swift 直接读取 ECN 协议栈状态进行离散判断，而 DCTCP 用标记比例 α 做连续调整；Swift 按信号类型施加差异化保留因子（0.70/0.75/0.50）而非统一的 $1 - \alpha/2$ ；Swift 同时利用丢包、ECN 与 CA 状态三类信号，且参数在线自适应。需要注意的是，DCTCP 的比例响应在标记密集时更平滑，而 Swift 的离散响应依赖 D_{max} 与冻结机制来避免过度收缩。

vs. BBR [?]: Swift 综合多种显式拥塞信号并以 $\alpha \times BDP$ 有界目标窗口维持稳态，BBR 依赖周期性的 pacing 增益扫描与最小 RTT 测量。在本次全矩阵数据中，BBR 在 36 个纯 TCP 场景的平均吞吐量为 Swift 的 94.7%（相对最强基线的逐场景增益 Swift 为 +2.1% ~ +5.8%），在微秒级 RTT 链路上亦正常运行（不再出现丢尾配置下的实现性退化），但在绝大多数场景略低于 Swift；在 UDP 突发下两者保持率相当（68.7% 对 68.6%）。BBR 的平均时延在多数场景为四协议最低（36 场景平均比 Swift 低约 0.8%），这与 BBR 维持较小 inflight 窗口的机制一致，本文如实记录。

5.3 适用边界与未来方向

在途水位与延迟的剩余代价：Swift 的吞吐增益来自以 $\alpha > 1$ （上界 1.30）维持的较高稳态在途水位，其在少数深缓冲或低速率场景中表现为相对延迟最优基线（通常是 BBR）最高约 10% 的平均时延抬升（如 wifi_legacy 71.22 ms 对 64.51 ms、congested_heavy 0.956 ms 对 NewReno 0.874 ms），绝对量为数十微秒至数毫秒。可探索的改进方向包

括：（1）引入显式排队时延估计 $q_{est} = RTT_{smoothed} - RTT_{min}$ 作为 α 调节的额外输入，实现排队延迟感知的 α 调度；（2）当 RTT 膨胀持续超过最小 RTT 感知阈值时把 α 收敛至 1.0 以下；（3）用多时间尺度 BDP 估计分离传播时延与排队时延。

参数预设依赖：保留因子（0.70/0.75/0.50）、 α 边界（[0.85, 1.30]）、连续递减阈值（ $D_{max} = 3$ ）与降窗后冻结的 ACK 事件数（4）均为手动调优的预设值。未来可通过元学习 [?] 或如 Gemini [?] 所示的带外贝叶斯优化实现参数的在线搜索，二者可与本文的每事件决策回路组合成分层结构。

深度/学习型策略增强：当前基于规则的启发式策略可解释性强但表示能力有限。后续若引入学习型组件（如离线强化学习训练、再经模型蒸馏提取规则），须在完整实验矩阵与异常过滤流程上重新评估，本文不将其作为当前系统的组成部分。

部署代价与真实环境验证：每个窗口调整事件一次跨进程往返是当前实现的主要部署障碍。后续需将决策逻辑内联为 Linux 内核 TCP 模块或 QUIC 用户态拥塞控制插件，并在覆盖无线/蜂窝接入、广域网与卫星链路的真实或半实物测试床上验证。仿真与真实环境的差异（NIC offload、中断合并、NUMA 效应、终端操作系统调度与无线链路波动）可能显著影响实际性能。

证据边界说明：本文全部定量结论的唯一数据来源是 logs/summary/kpi_forward.csv（288 条前向流口径记录）。该文件由 docs/plots/main.py 从 288 份 .flowmonitor 产物重新导出，本节与第 4 节的全部表格数值均直接取自该文件并按原始产物回溯核对。288 条记录全部通过完整性、物理相容性与取值域审计，无需排除任何场景或数据点，故本次更新未在 logs/error.txt 中新增排除记录。由此给出三条边界：其一，每配置仅一次确定性运行（单一随机种子），结论的稳健性建立在 36/36 的正增益方向一致性上，不附带跨种子置信区间；其二，结论适用范围为本文列出的 36 个场景与 RED/ECN 对称标记配置，队列管理策略、突发强度或场景集变化后需重新评估；其三，增益量级为百分之几（吞吐 +2% ~ +6%、丢包与公平性基本持平或略优），本文将其表述为“一致、无负向场景的小幅优势”，不夸大其量级。

6 结论

本文提出了 Swift——一种面向远距离传输与异构终端（手机、笔记本、台式机、边缘设备）的启发式多信号融合自适应拥塞控制算法：决策以确定性规则在每个 ACK/拥塞事件上于协议栈之外求解，不依赖训练样本与神经网络推理。其核心技术贡献收敛为三点：（1）多信号拥塞状态感知，把 ECN 协商与标记状态、拥塞避免状态机状态、拥塞事件语义、RTT 与最小 RTT、在途字节组织为 11 维有效观测子空间，并以两级优先级判定区分超时、ECN 触发的窗口收缩与普通丢包；（2）启发式反馈自适应的融合控制，由最小 RTT 感知的 RTT 膨胀比、反馈值相对其慢速基线的偏移与连续增长趋势共同调节每流 $\alpha \in [0.85, 1.30]$ ，并以有界步长逼近由滑动时间窗口投递速率最大值滤波得到的 $\alpha \times BDP$ 目

标窗口；（3）面向不确定拥塞信号的稳定性与安全机制，包括差异化保留因子（0.70/0.75/0.50）、连续递减下限 $D_{max} = 3$ 、降窗后 4 个 ACK 事件冻结、不迁就队列驻留的阈值更新、窗口钳位与超时丢弃陈旧动作。

在 ns-3.40 平台上，本文在 36 个场景 $\times$ 4 协议 $\times$ 2 设置共 288 次仿真的全矩阵（RED/ECN 对称标记、时间窗口带宽估计与基线相对反馈自适应的当前实现）上评估 Swift，全部指标仅按前向数据流定义并逐项回溯。结果显示：纯 TCP 设置下 Swift 在全部 36 个场景取得最高聚合吞吐量（相对最强基线 +2.1%~+5.8%、均值 +4.1%，相对 NewReno/CUBIC 均值 +6.0%），增益在微秒级近端高速、无线/蜂窝接入与远距离链路三类路径上均匀分布；平均单向时延相对 NewReno/CUBIC 均值低约 2.3%（36 个场景中 24 个更低），相对 BBR 平均高约 0.8%，个别深缓冲场景不超过 10%；四协议丢包率全部不超过 0.026%，Swift 的 Jain 指数均值 0.9989 为四者最高。UDP 突发下（平均负载 = 瓶颈 32%、峰值 64%）Swift 的绝对吞吐量仍在 36/36 个场景领先（+2.2%~+5.9%），吞吐保持率均值 68.6% 与基线持平，无任何协议发生塌缩。远距离场景受益明显：GEO 卫星纯 TCP 吞吐量 8.78 Mbps（相对 NewReno +7.5%）、突发下 5.65 Mbps（相对 NewReno +10.1%），LEO 卫星 134.95/93.58 Mbps、长途 WAN 881.2 Mbps 均为四协议最高。

本文同时如实披露混合结果与边界：Swift 的丢包率与 NewReno 相当、略低于 CUBIC/BBR，差值不超过 0.003 个百分点，本文不主张其“更不易丢包”；其在个别深缓冲场景的相对延迟抬升最高约 10%（绝对量数十微秒至数毫秒）；每配置为单一随机种子的确定性运行，结论以 36/36 的正增益方向一致性为主要支撑，不附带跨种子置信区间；结论适用于 RED/ECN 对称标记配置与本文列出的 36 个场景。后续工作包括：排队延迟感知的 α 调度、多时间尺度 BDP 估计、多种子重复实验、突发强度扫描，以及把每事件决策回路内联到内核或用户态协议栈以消除跨进程开销并在真实网络上验证。